

CHAPTER 02

数据的读取与处理

Data Reading and Processing

读取数据

数据概览

数据清洗

合并与透视

目录

CONTENTS

01 读取数据：CSV / Excel / 数据库

02 数据概览与描述性统计

03 数据清洗：重复 / 缺失 / 异常

04 数据合并与数据透视表

目录

CONTENTS

01 读取数据：CSV / Excel / 数据库 **本节**

02 数据概览与描述性统计

03 数据清洗：重复 / 缺失 / 异常

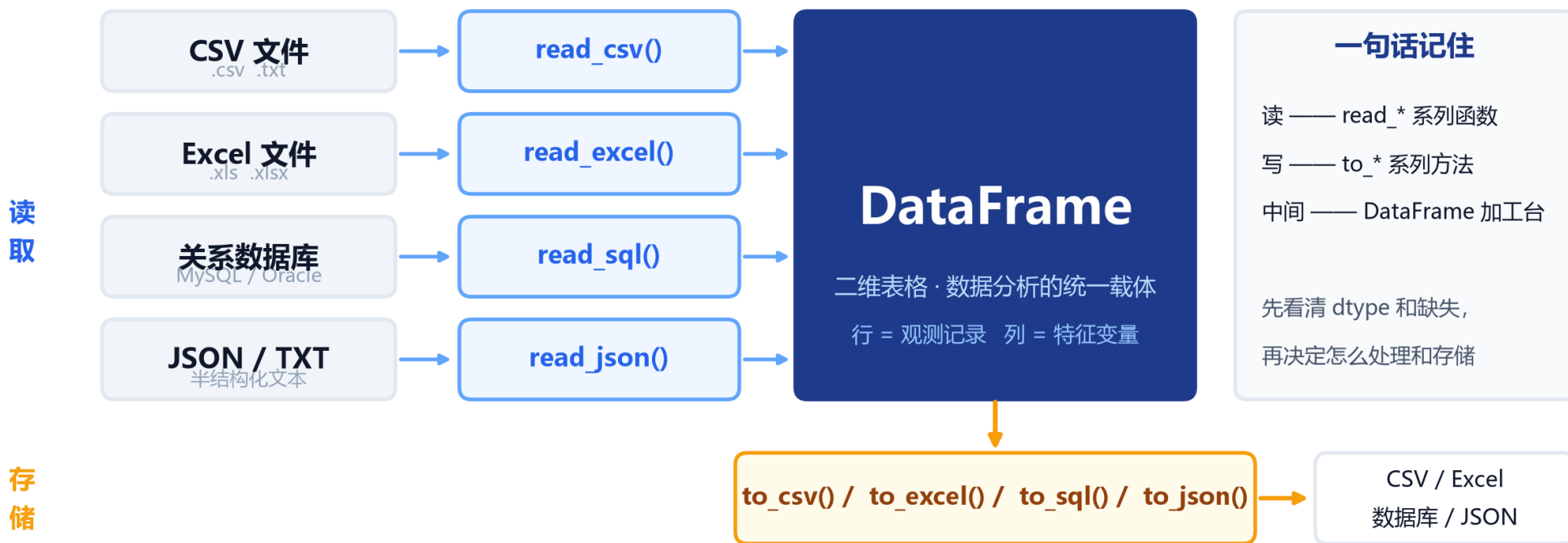
04 数据合并与数据透视表

读取数据

数据从哪里来 · pandas 读写体系

pandas 数据读写全景

读进来 · 加工好 · 存出去



读取 CSV 文件

1. read_csv() 读取文本数据

```
import pandas as pd

# 读取 CSV, 指定编码避免中文乱码
df = pd.read_csv("data/sales_2026.csv",
                  encoding="utf-8")

print(df.shape)      # 数据规模
print(df.head())     # 前 5 行
```

df.head() 输出

order_id	order_time	store	category	amount	qty
D260301001	2026-03-01 09:12	A店	家电	1280.0	2
D260301002	2026-03-01 09:35	B店	图书	158.0	1
D260301003	2026-03-01 10:02	A店	服饰	399.0	3
D260301004	2026-03-01 10:41	C店	家电	899.0	1
D260301005	2026-03-01 11:08	B店	食品	76.0	4

shape = (1200, 6) 共 1200 行 6 列

sep 默认是逗号; 中文文件若报编码错误, 先试 utf-8-sig, 再试 gbk

读取 CSV 文件

read_csv() 常用参数

读中文 CSV 最容易踩的坑是编码，其次是分隔符 —— 这两个参数先记牢。

参数名称	说明
filepath_or_buffer	接收 str，文件路径或 URL。无默认值
sep	接收 str，字段分隔符。默认为 ","
header	接收 int 或 sequence，指定某行作为列名。默认为 infer（自动识别）
names	接收 array，自定义列名。默认为 None
index_col	接收 int、sequence 或 False，指定索引列。默认为 None
usecols	接收 list，只读取指定列。默认为 None
dtype	接收 dict，指定列的数据类型。默认为 None
encoding	接收 str，文件编码。中文文件常用 "utf-8-sig" 或 "gbk"
nrows	接收 int，只读取前 n 行。默认为 None
chunksize	接收 int，分块读取，适合超大文件。默认为 None

存储 CSV 文件

2. to_csv() 写出到文本

```
to_csv() 存储 CSV

# 保存为 CSV
df.to_csv("output/sales_clean.csv",
          index=False,           # 不写出索引列
          encoding="utf-8-sig") # Excel 可直接打开

# 只保存部分列
df[["order_id", "amount"]].to_csv(
    "output/amount.csv", index=False)
```

index=False 是最常用的一个改动：
它决定要不要把行号写进文件。

写出中文时用 encoding="utf-8-sig",
Excel 双击打开才不会乱码。

存储 CSV 文件

to_csv() 常用参数

to_csv 与 read_csv 的参数高度对称，学会一个，另一个基本能迁移。

参数名称	说明
path_or_buf	接收 str，保存路径。无默认值
sep	接收 str，分隔符。默认为 ","
na_rep	接收 str，缺失值写出的占位符。默认为空字符串
columns	接收 list，指定写出的列。默认为 None（全部列）
header	接收 boolean，是否写出列名。默认为 True
index	接收 boolean，是否写出行索引。默认为 True（教学常改 False）
index_label	接收 sequence，索引列的名称。默认为 None
mode	接收 str，写入模式，w 覆盖 / a 追加。默认为 w
encoding	接收 str，编码格式。写出中文建议用 "utf-8-sig"

读取 Excel 文件

3. read_excel() 读取工作簿

```
import pandas as pd

# sheet_name 指定工作表, 0 表示第 1 张表
df = pd.read_excel("data/sales_2026.xlsx",
                  sheet_name=0,
                  header=0)      # 第 0 行作列名

# 一次读取全部工作表, 返回字典
all_sheets = pd.read_excel("data/sales_2026.xlsx",
                          sheet_name=None)

print(list(all_sheets.keys()))
```

一个工作簿可以有多张工作表。

sheet_name=0 表示第 1 张;
sheet_name="明细" 按名称指定;
sheet_name=None 一次性读全部,
返回一个 {表名: DataFrame} 字典。

读取 Excel 文件

read_excel() 常用参数

Excel 需要 **openpyxl** 引擎支持 .xlsx，老格式 .xls 需要 xlwt，请先在windows终端 安装库 **pip install openpyxl**

参数名称	说明
io	接收 str，文件路径或文件对象。无默认值
sheet_name	接收 str 或 int，指定工作表；None 读取全部（返回字典）。默认为 0
header	接收 int 或 sequence，指定列名所在行。默认为 0
names	接收 array，自定义列名。默认为 None
index_col	接收 int、sequence 或 False，指定索引列。默认为 None
usecols	接收 str、list 或 callable，只读取指定列。默认为 None
dtype	接收 dict，指定列的数据类型。默认为 None
skiprows	接收 int 或 list，跳过开头若干行。默认为 None
na_values	接收标量或 list，哪些值应被视为缺失。默认为 None

存储 Excel 文件

4. to_excel() 与 ExcelWriter

```
to_excel() 存储 Excel

# 写入 Excel, 指定工作表名
df.to_excel("output/sales_clean.xlsx",
            sheet_name="清洗后",
            index=False)

# 一张工作簿写入多张表
with pd.ExcelWriter("output/report.xlsx") as w:
    df_a.to_excel(w, sheet_name="A店", index=False)
    df_b.to_excel(w, sheet_name="B店", index=False)
```

to_excel 与 to_csv 有三点差异:

- ① 路径参数叫 excel_writer;
- ② 没有 sep 参数;
- ③ 多了 sheet_name 参数。

写多张表必须用 ExcelWriter 上下文管理器, 否则后写的会覆盖前面的。

存储 Excel 文件

为什么要用 ExcelWriter

- 写一张表: `df.to_excel("out.xlsx", index=False)` 即可
- 写多张表: 必须借助 `pd.ExcelWriter` 上下文管理器
- 数据量大时 Excel 有 104 万行上限, 超出需改用 CSV 或数据库
- 写好的 Excel 若要用作汇报素材, 建议同时设置 `index=False` 和 `sheet_name`

```
with pd.ExcelWriter("output/report.xlsx") as w:  
    df_a.to_excel(w, sheet_name="A店", index=False)  
    df_b.to_excel(w, sheet_name="B店", index=False)
```

读取数据库数据

5. read_sql() 与 SQLAlchemy

```
read_sql() 读取数据库

import pandas as pd
from sqlalchemy import create_engine

# 1) 建立数据库连接
engine = create_engine(
    "mysql+pymysql://root:123456@localhost:3306/
    shop?charset=utf8")

# 2) 用 SQL 查询读取
sql = "SELECT * FROM sales WHERE amount > 500"
df = pd.read_sql(sql, con=engine)
```

三个读取函数的区别：

read_sql_table 只能读整张表

read_sql_query 只能执行 SQL 查询

read_sql 两者综合，最常用

con 既可以传 Engine，
也可以传已建立的 Connection。

读取数据库数据

连接字符串的拼写规则

- 格式：数据库产品名+驱动名://**用户名:密码@IP地址:端口号**/数据库名
- MySQL 用 mysql+pymysql://... 并需先安装 pymysql 驱动
- Oracle 用 oracle+cx_oracle://... 并需安装 cx_Oracle
- 中文库建议在末尾加上 ?charset=utf8，否则可能读出乱码
- 连接字符串写错是最常见的报错来源，建议拆成变量逐步拼接再调试

```
conn_str = "mysql+pymysql://root:123456@localhost:3306/shop?charset=utf8"  
engine = create_engine(conn_str)
```

读取数据库数据

read_sql() 常用参数

数据库读取的返回结果同样是 DataFrame，后续处理方式完全一致。

参数名称	说明
sql	接收 str，要读取的表名或 SQL 语句。无默认值
con	接收数据库连接（Engine 或 Connection）。无默认值
index_col	接收 int、sequence 或 False，指定作为行索引的列。默认为 None
coerce_float	接收 boolean，是否把 decimal 转成 float64。默认为 True
parse_dates	接收 list 或 dict，指定需要解析为日期的列。默认为 None
columns	接收 list，指定读取的列名。默认为 None
chunksize	接收 int，分块返回迭代器，适合大表。默认为 None

存储到数据库

6. to_sql() 写出到数据表

```
to_sql() 存储数据库

# 写入数据库表
df.to_sql("sales_clean",          # 表名
          con=engine,
          if_exists="append",     # 追加而非重建
          index=False)

# if_exists 三种取值
# fail      表已存在则报错（默认）
# replace   删掉原表后重建
# append    在原有数据后追加
```

if_exists 是 to_sql 最需要注意的参数：

fail 表存在则报错（默认）

replace 删掉原表后重建

append 在原有数据后追加

生产环境慎用 replace ——
它会删除原有数据！

本节小结

三种数据源的读写对照

- 文件类数据（CSV / Excel）：read_* 读入，to_* 写出，参数以编码、分隔符、工作表为主
- 数据库数据：先 create_engine 建立连接，再用 read_sql 读、to_sql 写
- 所有读取函数返回的都是 DataFrame —— 后续所有处理都在这个对象上完成
- 读取完成后的第一件事不是分析，而是做一次数据概览，确认类型与缺失情况

读：read_csv / read_excel / read_sql → DataFrame → 写：to_csv / to_excel / to_sql

目录

CONTENTS

01 读取数据：CSV / Excel / 数据库

02 数据概览与描述性统计

本节

03 数据清洗：重复 / 缺失 / 异常

04 数据合并与数据透视表

数据概览

拿到数据先看清全貌，再决定怎么处理

拿到数据后的"概览五件套"

先看清数据全貌，再决定怎么处理

看前 5 行

head()

快速确认结构
是否符合预期

看行列数

shape

(行数, 列数)
掌握数据规模

看列与类型

info()

列名 / 非空计数
dtype 一目了然

看统计摘要

describe()

均值 标准差
四分位数 极值

看缺失情况

isnull().sum()

逐列统计
缺失值数量

数据概览

1. shape / columns / info / dtypes

```
print(df.shape)      # (1200, 6) 行数与列数
print(df.columns)    # 列名
print(df.index)      # 行索引

df.info()            # 列名 / 非空计数 / dtype
df.dtypes            # 每一列的数据类型
df.isnull().sum()    # 每一列的缺失值个数
```

df.info() 输出

#	column	non-null	dtype
0	order_id	1200	object
1	order_time	1200	object
2	store	1194	object
3	category	1200	object
4	amount	1158	float64
5	quantity	1200	int64

store 缺 6 条, amount 缺 42 条

info() 输出里的 non-null 计数, 就是发现缺失值的第一条线索

描述性统计

2. describe() 与常用统计量

```
describe() 描述性统计

# 数值列的描述性统计
print(df[["amount", "quantity"]].describe())

# 单独计算常用统计量
print(df["amount"].mean())      # 均值
print(df["amount"].median())    # 中位数
print(df["amount"].std())       # 标准差
print(df["amount"].quantile([0.25, 0.75]))

# 类别列的取值分布
print(df["category"].value_counts())
```

describe() 输出

	amount	quantity
count	1158.00	1200.00
mean	567.37	2.31
std	667.01	1.52
min	12.00	1.00
25%	168.75	1.00
50%	398.00	2.00
75%	762.50	3.00
max	9980.00	9.00

max 9980 远高于 75% 分位 762.5 → 优先排查

describe() 默认只统计数值列；要看类别列需加 include="object"

描述性统计

统计量怎么读

- count 非空计数 —— 与总行数相减，差额就是缺失值个数
- mean 均值 与 50% 中位数 —— 两者相差悬殊，说明分布偏斜或存在异常值
- std 标准差 —— 衡量数据离散程度，数值越大说明波动越剧烈
- 25% / 75% 四分位数 —— 配合中位数判断分布形态，也是 IQR 异常值检测的基础
- min / max 极值 —— 最直接的异常值筛查入口，但一定要结合业务判断

本案例：max = 9980，而 75% 分位数只有 762.5 → 最大值远超整体水平，优先排查

数据类型与转换

类型读错，后面的统计和绘图都会跟着出错

dtype 与类型转换 astype()

类型读错，后面的统计和绘图都会跟着出错

dtype	含义	典型场景	常见转换
object	字符串 / 混合	门店名、类别、误入的数字	astype("category")
int64	整数	订单量、商品数量	astype("float64")
float64	浮点数	金额、比率（可含 NaN）	astype("int64")
datetime64	日期时间	下单时间 pd.to_datetime	dt.year / dt.month
category	分类	性别、等级、地区	astype("str")
bool	布尔	是否支付、是否退货	astype("int")

数据类型与转换

3. astype() 与 to_datetime()

```
astype() 类型转换

# 时间字符串 → datetime64
df["order_time"] = pd.to_datetime(df["order_time"])

# 从时间中抽取年 / 月 / 星期
df["month"] = df["order_time"].dt.month
df["weekday"] = df["order_time"].dt.dayofweek

# 整数 → 浮点 (后续可能产生 NaN)
df["quantity"] = df["quantity"].astype("float64")

# 低频字符串列 → 分类, 节省内存
df["category"] = df["category"].astype("category")
```

三个最容易出问题的转换:

- ① 日期字符串 → datetime64
用 `pd.to_datetime()`, 转完才能用 `.dt`
取年、月、星期
- ② 数字被读成字符串
常见于带千分位或货币符号, 需先
清洗再 `astype`
- ③ `float` → `int` 会报错
因为 `float64` 里有 `NaN`, 必须先
处理缺失再转 `int64`

数据类型与转换

关于类别型 category

- 把重复度高的字符串列转成 category，内存占用可下降 80% 以上
- category 类型可以自定义顺序，例如"低 < 中 < 高"，便于排序与绘图
- 但 category 不能直接做数值运算，需要先 `astype("str")` 或映射为数字
- 时间列转为 `datetime64` 之后，才可以用 `resample` 做重采样和时序分析

```
df["category"] = df["category"].astype("category")    # 省内存，但不参与数值运算
```

本节小结

概览与统计的四步走

- 第一步 看形状：shape 确认行列数，判断数据是否读全了
- 第二步 看类型：info() / dtypes 确认每列 dtype 是否符合预期
- 第三步 看统计：describe() 快速识别离群值、量级差异与分布形态
- 第四步 看分类：value_counts() 检查类别列是否存在拼写不一致的脏数据

概览不是走过场 —— 它决定了后面清洗策略的选择，也决定了可视化能画出什么

目录

CONTENTS

01 读取数据：CSV / Excel / 数据库

02 数据概览与描述性统计

03 数据清洗：重复 / 缺失 / 异常

本节

04 数据合并与数据透视表

数据质量问题

重复值 · 缺失值 · 异常值

数据质量的三大典型问题

重复值

同一条记录被多次采集



方差变小 · 分布失真

缺失值

字段信息为空
MCAR / MAR / MNAR



样本减少 · 分析产生偏差

异常值

明显偏离其余观测值的个别值



产生“伪回归” · 歪曲结论

数据清洗决策流程

先检测、再定位、后处理、最后复核

数据清洗决策流程

先检测、再定位、后处理



原则：能补不删、能留不删；每次处理都要记录原因，并在处理后重新做一次数据概览

校验数据：一致性

多源数据合并前必须先对齐

一致性校验的四个常见维度

多源数据合并前必须先对齐

时间范围

不同表的时间取值区间不同
例：3/2-3/29 vs 3/15-4/18

处理：补全时间字段

时间粒度

采集频率不一致
例：1 分钟 vs 30 秒

处理：重采样至统一粒度

时间格式

字符串格式不统一
例：2020-09-01 vs 20200901

处理：统一 to_datetime

时区

服务器时区未修改
例：本地 12:12 vs 海外 17:12

处理：统一时区转换

校验数据：字段信息

同名异义与异名同义，是合并失败的两大隐藏原因

字段信息校验：同名异义 与 异名同义

同名异义 —— 字段名相同，含义不同

数据源A · ID	含义	数据源B · ID	含义
2003117399	商品编号	1014000141	订单编号

→ 直接按 ID 合并会张冠李戴，必须先重命名字段

异名同义 —— 字段名不同，含义相同

A.sales_dt	销售日期	B.sales_date	销售日期
2020/3/02	2020/3/02	2020/3/02	2020/3/02

→ A.sales_dt = B.sales_date，统一命名后可正常合并

校验数据：缺失值

1. isnull() / notnull() / count()

```
# 逐元素判断是否缺失，返回布尔表
df.isnull().head()

# 逐列统计缺失个数与占比
miss = df.isnull().sum()
ratio = miss / len(df) * 100
print(pd.DataFrame({"缺失数": miss, "占比%": ratio}))

# 定位含缺失的行
df[df["amount"].isnull()]
```

缺失情况统计

column	缺失数	占比%
order_id	0	0.00
order_time	0	0.00
store	6	0.50
category	0	0.00
amount	42	3.50
quantity	0	0.00

占比 < 5% 可考虑删除；占比高需谨慎

先量化再决策：占比低可以考虑删除，占比高则需要慎重分析

校验数据：缺失机制

判断缺失机制，才能选对处理方法

缺失值的三种分布机制

判断缺失机制，才能选对处理方法

MCAR

完全随机缺失

缺失与任何变量都无关
可直接删除，影响最小

删除法

MAR

随机缺失

缺失依赖于其他已观测变量
可借助其他变量填补

替换法 / 模型预测

MNAR

完全非随机缺失

缺失依赖于自身未观测的值
删除会引入偏差，需谨慎

插值法 / 保留标记

清洗数据：删除法

2. dropna() 丢弃缺失记录

```
dropna() 删除法

# 删除法：丢弃含缺失的行
df_drop = df.dropna(axis=0, how="any")

# 只在指定列上检查缺失
df_drop = df.dropna(subset=["amount"])

# 删除缺失比例过高的列（如超过 50%）
thresh = int(len(df) * 0.5)
df_drop = df.dropna(axis=1, thresh=thresh)

print("删除前:", df.shape, "删除后:", df_drop.shape)
```

删除法的代价是样本量减少，适用于：

- 缺失占比低（一般 < 5%）
- 缺失属于完全随机（MCAR）
- 该列不是分析的关键变量

如果缺失并非随机，删除会引入系统性偏差，此时应改用填充。

清洗数据：删除法

dropna() 常用参数

dropna 是"减少样本量换信息完整度"的方法，用之前一定要先量化缺失占比。

参数名称	说明
axis	接收 0 或 1。0 删除含缺失的行，1 删除含缺失的列。默认为 0
how	接收 "any" 或 "all"。any 有一个缺失就删，all 全部缺失才删。默认为 any
thresh	接收 int，保留至少 thresh 个非缺失值，否则删除该行/列。默认为 None
subset	接收类 array，只在指定列/行上检查缺失。默认为 None（全部）
inplace	接收 boolean，是否在原表上修改。默认为 False

清洗数据：替换法

3. fillna() 用特定值填补

```
fillna() 替换法

# 数值列：用均值 / 中位数填充
df["amount"] = df["amount"].fillna(df["amount"].mean())

# 类别列：用众数填充
mode = df["store"].mode()[0]
df["store"] = df["store"].fillna(mode)

# 前后向填充（适合时间序列）
# 注：pandas 3.x 起 fillna(method=...) 已移除
df["amount"] = df["amount"].ffill() # 沿用上一个有效值
df["amount"] = df["amount"].bfill() # 使用下一个有效值

# 不同列用不同值填充
df = df.fillna({"amount": 0, "store": "未知"})
```

填充值怎么选：

- 数值型 → 均值 / 中位数
- 类别型 → 众数
- 时间序列 → 前向填充 ffill

注意：填充会改变数据的标准差，从而影响信息量。因此处理后要在报告中说明填补方式。

清洗数据：替换法

fillna() 常用参数

不同列可以填不同的值：`df.fillna({"amount": 0, "store": "未知"})`

参数名称	说明
value	接收 scalar、dict、Series 或 DataFrame，用于替换的值。无默认值
method	接收 "ffill"/"pad"（沿用上一个有效值）或 "bfill"/"backfill"（用下一个）。默认为 None
axis	接收 0 或 1，填充方向。默认为 0
inplace	接收 boolean，是否在原表上修改。默认为 False
limit	接收 int，单次最多连续填补的个数上限。默认为 None

清洗数据：插值法

4. interpolate() 按趋势推算

```
interpolate() 插值法

# 线性插值：按前后已知值等比推算
df["amount"] = df["amount"].interpolate(
    method="linear")

# 时间序列插值：按时间间隔加权
df = df.set_index("order_time")
df["amount"] = df["amount"].interpolate(method="time")

# method 常用取值
#   linear   线性插值（默认）
#   time     按时间加权      polynomial 多项式
#   spline   样条插值        pad         沿用前一个值
```

三种插值思路：

线性插值 linear

假定两点间线性变化，按距离等比推算

多项式插值 polynomial

用多项式拟合已知点，阶数过高会震荡

样条插值 spline

分段多项式，连接处导数连续，更光滑

清洗数据：插值法

三种方法怎么选

- 删除法：实现最简单，但样本减少，可能破坏数据结构
- 替换法：实现也简单，但会改变标准差，导致信息量变动
- 插值法：更贴近数据原有趋势，适合有序数据和时间序列
- 时间序列务必用 `method="time"`，它按时间间隔加权而非按行号等距推算

```
df["amount"] = df["amount"].interpolate(method="linear") # 最常用
```

清洗数据：重复值

5. duplicated() 与 drop_duplicates()

```
drop_duplicates() 去重

# 统计完全重复的行数
print(df.duplicated().sum())

# 删除重复行，保留第一次出现的
df = df.drop_duplicates(keep="first")

# 按关键列去重（订单号不允许重复）
df = df.drop_duplicates(subset=["order_id"],
                        keep="last")

# 去重后务必重置索引
df = df.reset_index(drop=True)
```

记录重复 vs 特征重复

记录重复：某些行完全相同
→ drop_duplicates()

特征重复：列名不同但数据相同
→ 用 corr() 算相关系数，
相似度接近 1 的去掉一个

去重后记得 reset_index(drop=True)

清洗数据：重复值

drop_duplicates() 常用参数

业务上还有一种"伪重复"：同一订单因多次操作产生多条流水，需要先按业务规则排序再 keep="last"。

参数名称	说明
subset	接收 str 或 sequence，只在指定列上判断重复。默认为 None（全部列）
keep	接收 "first"（保留第一条）、"last"（保留最后一条）、False（重复全删）。默认为 first
inplace	接收 boolean，是否在原表上修改。默认为 False
ignore_index	接收 boolean，去重后是否重建索引。默认为 False

清洗数据：特征重复

6. corr() 识别冗余特征

```
corr() 识别特征重复

# 计算特征间的相关系数矩阵
corr = df[["amount", "quantity", "unit_price"]].corr()

# 相似度接近 1 的两个特征可去掉一个
print(corr)

# 斯皮尔曼秩相关 (不要求正态分布)
corr_sp = df[["amount", "quantity"]].corr(
    method="spearman")
```

相关系数矩阵

	amount	quantity	unit_price
amount	1.000	0.912	0.998
quantity	0.912	1.000	0.905
unit_price	0.998	0.905	1.000

amount 与 unit_price 相关 0.998 → 高度冗余

相关系数接近 1 的两个特征信息高度重叠，保留一个即可

清洗数据：异常值检测

7. 3σ 原则与 IQR 准则

```
异常值检测

# 方法一:  $3\sigma$  原则 (近似正态分布)
mu, sigma = df["amount"].mean(), df["amount"].std()
cond = (df["amount"] > mu + 3*sigma) | \
        (df["amount"] < mu - 3*sigma)
print("3 $\sigma$  异常:", df[cond].shape[0], "条")

# 方法二: IQR 四分位距 (不要求正态分布)
q1, q3 = df["amount"].quantile([0.25, 0.75])
iqr = q3 - q1
cond = (df["amount"] > q3 + 1.5*iqr) | \
        (df["amount"] < q1 - 1.5*iqr)
print("IQR 异常:", df[cond].shape[0], "条")
```

两种方法怎么选:

3σ 原则

要求数据近似正态分布;
超出 $\mu \pm 3\sigma$ 的仅占 0.27%,
判定为高度异常。

IQR 准则

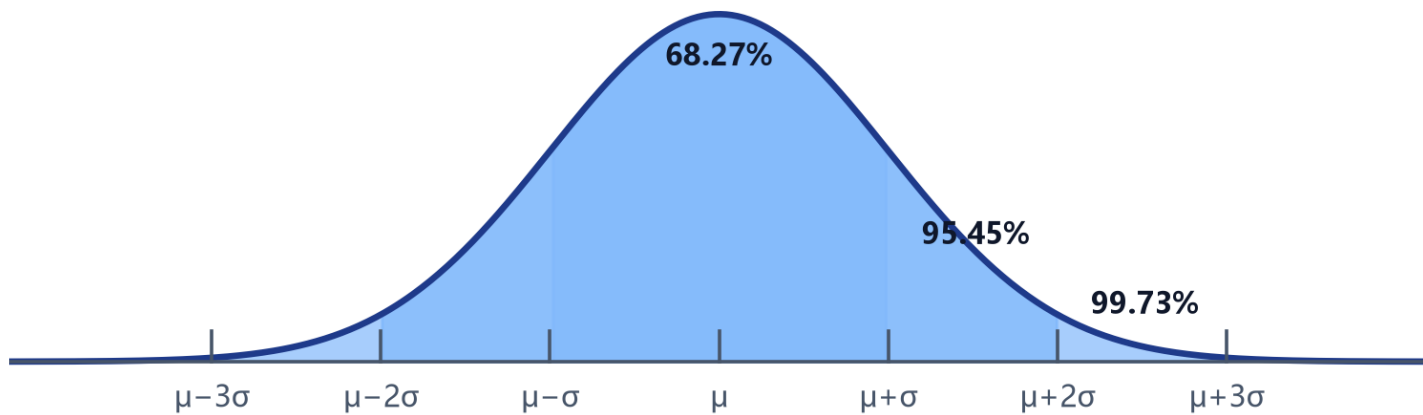
不要求正态分布, 更稳健;
用四分位数划定上下界,
是箱线图的判定依据。

清洗数据：异常值检测

3 σ 原则：适用于近似正态分布的连续型特征

3 σ 原则（拉依达准则）

数据近似正态分布时，超出 $\mu \pm 3\sigma$ 的概率仅约 0.27%



判定标准

$$|x - \mu| > 2\sigma$$

疑似异常值

$$|x - \mu| > 3\sigma$$

高度异常值

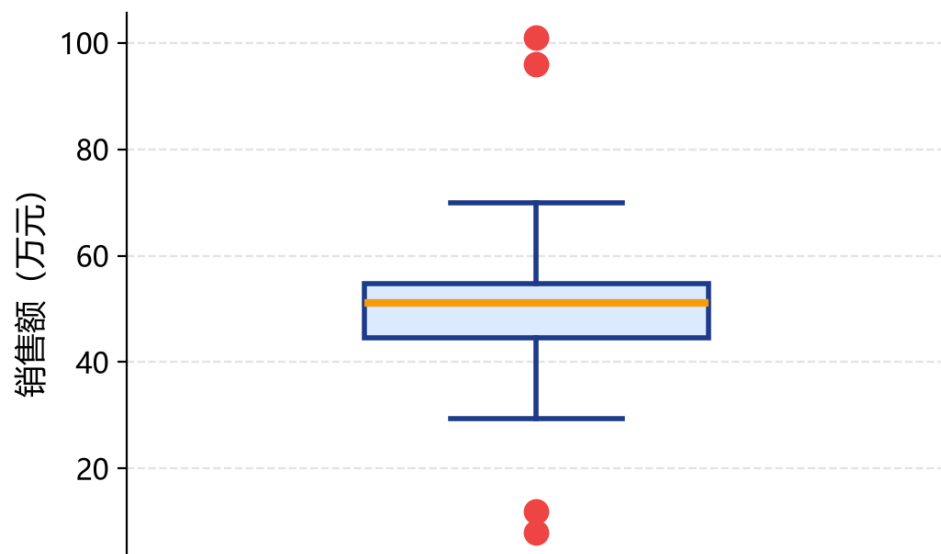
适用于近似正态分布的
连续型数值特征

清洗数据：异常值检测

IQR 准则：不要求正态分布，更稳健

箱线图与四分位距 (IQR) 准则

$IQR = Q3 - Q1$, 落在 $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ 之外即为异常值



判定规则

下四分位数 $Q1$	44.5
上四分位数 $Q3$	54.8
四分位距 $IQR = Q3 - Q1$	10.3
下界 $Q1 - 1.5 \times IQR$	29.1
上界 $Q3 + 1.5 \times IQR$	70.2

落在区间之外的点 → 判定为异常值 (图中红点)

清洗数据：异常值处理

8. 四种处理方式

方式一：删除含异常值的记录

```
df = df[~cond]
```

方式二：视为缺失值，再按缺失值处理

```
df.loc[cond, "amount"] = np.nan
```

```
df["amount"] = df["amount"].fillna(df["amount"].median())
```

方式三：用边界值截断（盖帽法）

```
df["amount"] = df["amount"].clip(q1 - 1.5*iqr, q3 + 1.5*iqr)
```

方式四：保留（异常值本身可能就是业务重点）

选哪种，取决于异常值的成因：

数据录入错误（读错、记错、单位错）

→ 删除或修正

系统采集异常（传感器跳变、bug）

→ 视为缺失值再处理

业务真实存在（大额订单、爆款）

→ 必须保留，这恰恰是重点

清洗数据：异常值处理

一条重要原则

- 异常值不等于错误值 —— 很多业务洞察恰恰藏在异常值里
- 删除异常值前，先问一句：这个值在业务上讲得通吗？
- 如果拿不准，就做两版分析：保留异常值一版、剔除一版，对比结论是否一致
- 处理前后都要打印 `shape` 和 `describe()`，确认没有引入新问题

`clip()` 盖帽法： `df["amount"].clip(q1 - 1.5*iqr, q3 + 1.5*iqr)` —— 保留记录但压缩极值

本节小结

数据清洗检查清单

- 重复值: duplicated() 定位 → drop_duplicates() 去重 → reset_index() 重建索引
- 缺失值: isnull() 量化 → 判断缺失机制 → 删除 / 替换 / 插值 三选一
- 异常值: 箱线图或 3σ 检测 → 结合业务判断成因 → 删除 / 修正 / 保留
- 合并前: 一致性校验 (时间、字段) 必须在合并之前完成, 否则会报错或产生大量空值

清洗没有标准答案 —— 判断依据是业务逻辑, 且每一次处理都要能被复现和解释

目录

CONTENTS

01 读取数据：CSV / Excel / 数据库

02 数据概览与描述性统计

03 数据清洗：重复 / 缺失 / 异常

04 数据合并与数据透视表

本节

合并数据

为什么需要合并

- 数据种类繁多，不同主题的数据往往分别存放在不同的表里
- 数据量大时，会按时间段拆分存储，表的数量随时间不断增加
- 把有关联的数据整合到一张表，后续的可视化与分析效率会大幅提高
- 合并方法分三类：堆叠合并、主键合并、重叠合并；数据透视表则用于重塑数据视角

堆叠 = 拼在一起 | 主键合并 = 按 key 对齐 | 重叠合并 = 互相补齐缺口

堆叠合并：横向

concat(axis=1): 列数增加，行数不变

横向堆叠：concat(axis=1)

沿 X 轴按行索引对齐拼接，列数增加，行数不变
合并结果

df1 (销售额)

df2 (订单量)

门店	销售额
A店	120
B店	95
C店	138

+

门店	订单量
A店	32
B店	27
C店	41



门店	销售额	订单量
A店	120	32
B店	95	27
C店	138	41

关键参数

axis = 1 横向拼接

join = outer / inner

objs = 待合并对象的列表

行索引对齐: join="outer" 取并集，缺失补 NaN; join="inner" 只保留交集

堆叠合并：横向

1. concat(axis=1) 按行索引对齐

```
concat(axis=1) 横向堆叠

import pandas as pd

# 横向堆叠: axis=1, 按行索引对齐
result = pd.concat([df_amount, df_quantity],
                    axis=1, join="outer")

# 只保留索引完全重叠的部分
inner = pd.concat([df_amount, df_quantity],
                  axis=1, join="inner")

print(result.shape)  # 列数增加, 行数不变
```

axis=1 时 concat 做行对齐:

join="outer" (默认)
取索引并集, 缺失处补 NaN

join="inner"
只返回索引重叠的部分

横向堆叠不要求列名一致,
但要求行索引能对得上。

堆叠合并

concat() 常用参数

join 参数决定了索引不完全一致时，是取交集还是取并集。

参数名称	说明
objs	接收 Series / DataFrame 的列表或字典，表示参与连接的对象。无默认值
axis	接收 0 或 1。0 纵向堆叠，1 横向堆叠。默认为 0
join	接收 "inner" 或 "outer"。按其他轴索引取交集还是并集。默认为 outer
ignore_index	接收 boolean，是否重建连接轴上的索引。默认为 False
keys	接收 sequence，为各段数据添加层次化索引标签，便于区分来源。默认为 None
names	接收 list，为 keys 创建的层级命名。默认为 None
verify_integrity	接收 boolean，是否检查新轴上的索引重复，重复则报错。默认为 False
sort	接收 boolean，是否对非连接轴排序。默认为 False
copy	接收 boolean，是否复制数据。默认为 True

堆叠合并：纵向

concat(axis=0): 行数增加，列数不变

纵向堆叠：concat(axis=0) / append()

df1 (3月)

门店	销售额
A店	120
B店	95

df2 (4月)

门店	销售额
C店	138
D店	76

+

沿 Y 轴按列名对齐拼接，行数增加，列数不变

合并结果 (追加行)

门店	销售额
A店	120
B店	95
C店	138
D店	76

两种写法

```
pd.concat([df1, df2], axis=0)
```

```
df1.append(df2)
```

- 列名不一致时 outer 取并集
- append 要求列名完全一致
- ignore_index=True 可重建索引
- keys=[...] 可标记数据来源

堆叠合并：纵向

2. concat(axis=0) 纵向堆叠

```
concat(axis=0) 纵向堆叠

# 纵向堆叠: axis=0, 按列名对齐 (默认)
result = pd.concat([df_mar, df_apr], axis=0)

# 重建索引, 避免索引重复
result = pd.concat([df_mar, df_apr],
                    axis=0, ignore_index=True)

# 标记每段数据来源
result = pd.concat([df_mar, df_apr],
                    keys=["3月", "4月"])

# 老写法 df1.append(df2) 在 pandas 3.x 已移除

print(result.shape) # 行数增加, 列数不变
```

axis=0 时 concat 做列对齐:

列名一致 → 直接拼接

列名不同 → outer 取并集补 NaN,
inner 只保留公共列

append() 是 concat 的简化版,
要求两张表列名完全一致,
但 pandas 3.x 起已被移除, 请统一改用 concat。

主键合并

merge(): 按 key 把两张表的行连接起来

主键合并: merge() / join()

按一个或多个键 (key) 把两张表的行连接起来, 类似 SQL 的 JOIN
how="outer" 全外连接

左表 left

商品ID	销售额
P01	120
P02	95
P03	138

右表 right

商品ID	类别
P01	家电
P02	图书
P04	服饰

on = 商品ID

商品ID	销售额	类别
P01	120	家电
P02	95	图书
P03	138	NaN
P04	NaN	服饰

未匹配上的位置自动补 NaN

how 参数四种连接

- inner** 仅保留键的交集
- left** 保留左表全部行
- right** 保留右表全部行
- outer** 保留键的并集

主键合并

3. merge() 按主键连接

```
merge() 主键合并

# 按商品ID 主键做内连接
df = pd.merge(df_sales,          # 左表
               df_product,       # 右表
               on="product_id",  # 连接键
               how="inner")      # 连接方式

# 左右表主键名不同时
df = pd.merge(df_sales, df_product,
               left_on="pid", right_on="product_id",
               how="left")

# 重名列自动加后缀 _x / _y
df = pd.merge(df_a, df_b, on="key",
               suffixes=("_a", "_b"))
```

on 与 left_on / right_on:

键名相同 → 用 on="key"

键名不同 → left_on / right_on
分别指定

重名列会自动加后缀 _x / _y,
也可以用 suffixes 自定义。

主键合并

merge() 常用参数

how 是最关键的一个参数：它决定了保留哪些行、丢弃哪些行。

参数名称	说明
left / right	接收 DataFrame，分别表示左表和右表。无默认值
how	接收 inner / outer / left / right，表示连接方式。默认为 inner
on	接收 str 或 sequence，左右表同名的连接键。默认为 None
left_on / right_on	接收 str 或 sequence，左右表键名不同时分别指定。默认为 None
left_index / right_index	接收 boolean，是否用索引作为连接键。默认为 False
suffixes	接收 tuple，重名列的后缀。默认为 ("_x", "_y")
sort	接收 boolean，是否按连接键排序。默认为 False
indicator	接收 boolean 或 str，添加一列标记每行的来源。默认为 False

主键合并

4. join() 按索引合并

```
join() 索引合并

# join 按索引合并, 要求主键名相同
df_sales.join(df_product,
              on="product_id",
              how="left",
              lsuffix="_s", rsuffix="_p")

# merge 与 join 的区别
#   merge: 任意列做键, 左右键名可不同, 功能更全
#   join : 以索引为主, 写法更简洁
#   两者都能实现 inner / left / right / outer
```

merge 与 join 怎么选:

merge 功能更全

任意列做键、左右键名可不同、
支持 indicator 标记来源

join 写法更简洁

以索引为默认键, 适合已
set_index 的表; 但要求
两侧键名一致

主键合并

merge() 与 join() 的三个区别

- 连接的键不同：merge 默认按列，join 默认按索引
- 键名要求不同：merge 允许左右键名不同，join 要求一致
- 功能范围不同：merge 支持 indicator、validate 等更多参数
- 实际工作中 merge 用得更多，join 适合链式写法 `df1.join(df2).join(df3)`

```
df_sales.join(df_product, on="product_id", how="left", lsuffix="_s", rsuffix="_p")
```

重叠合并

`combine_first()`: 用另一张表补齐当前表的缺口

重叠合并: `combine_first()`

df1 (有缺口)

门店	销售额
A店	120
B店	NaN
C店	NaN

+

df2 (补全数据) 用另一张表的数据补齐当前表的缺失值 按索引对齐

门店	销售额
B店	95
C店	138



门店	销售额
A店	120
B店	95
C店	138

补全结果

补齐规则

- df1 有值 → 保留 df1 的值
- df1 缺失 → 取 df2 的值
- 两者皆缺 → 结果仍为 NaN

重叠合并

5. combine_first()

```
combine_first() 重叠合并

# 用 df2 的数据补齐 df1 的缺失值
df = df1.combine_first(df2)

# 等价写法: 按列精确填充
df1["amount"] = df1["amount"].fillna(
    df2["amount"])

# 注意: combine_first 按索引对齐
df2 = df2.set_index("order_id")
df = df1.set_index("order_id").combine_first(df2)
```

适用场景:

两份数据内容几乎一致,
但某些特征在 A 表完整、
在 B 表缺失, 反之亦然。

补齐规则:

当前表有值 → 保留当前表
当前表缺失 → 取另一张表
两者皆缺 → 仍为 NaN

数据透视表

pivot_table(): 长表 → 宽表, 重塑数据观察视角

数据透视表: pivot_table()

长表 → 宽表: 按行/列分组聚合, 重塑数据观察视角

原始长表 (每行一条记录)

门店	季度	销售额
A店	Q1	120
A店	Q2	135
B店	Q1	95
B店	Q2	108
C店	Q1	138
C店	Q2	126

`pivot_table()`



透视结果 (门店 × 季度)

门店	Q1	Q2
A店	120	135
B店	95	108
C店	138	126

核心参数

index	行分组键 (如 门店)
columns	列分组键 (如 季度)
values	被聚合的数值列
aggfunc	聚合方式, 默认 mean
fill_value	缺失值填充值
margins	是否显示合计行列

数据透视表

6. pivot_table() 分组透视

```
pivot_table() 透视表

# 门店 × 季度 的销售额透视表
pv = df.pivot_table(index="store",
                    columns="quarter",
                    values="amount",
                    aggfunc="sum",
                    fill_value=0,
                    margins=True) # 显示合计

print(pv)

# 多重分组透视
pv2 = df.pivot_table(index=["store", "category"],
                    columns="quarter", values="amount",
                    aggfunc=["sum", "mean"])
```

透视结果 (单位: 万元)

quarter	A店	B店	C店	All
Q1	86.4	52.1	73.8	212.3
Q2	94.7	58.3	81.2	234.2
Q3	102.5	61.9	88.6	253.0
Q4	128.3	75.4	104.1	307.8
All	411.9	247.7	347.7	1007.3

margins=True 自动汇总行与列

margins=True 自动生成"合计"行与列, 汇报时非常实用

数据透视表

pivot_table() 常用参数

index 定行、columns 定列、values 定值、aggfunc 定怎么算 —— 记住这四要素。

参数名称	说明
data	接收 DataFrame，被透视的原始数据。默认为当前 DataFrame
index	接收 str、list 或 array，行分组键（透视表的行）。无默认值
columns	接收 str、list 或 array，列分组键（透视表的列）。无默认值
values	接收 str、list 或 array，被聚合的数值列。默认为全部数值列
aggfunc	接收函数、函数列表或 dict，聚合方式。默认为 mean
fill_value	接收 scalar，结果中缺失值的填充值。默认为 None
margins	接收 boolean，是否添加"合计"行与列。默认为 False
dropna	接收 boolean，是否删除全为缺失的列。默认为 True
observed	接收 boolean，类别变量是否只显示出现过的组合。默认为 False

分组聚合

7. groupby() 与 agg()

```
groupby() 分组聚合

# 按门店分组，计算销售额的多种统计
g = df.groupby("store")["amount"].agg(
    ["count", "sum", "mean", "max"])

# 多字段分组 + 自定义列名
g = df.groupby(["store", "category"]).agg(
    订单量=("order_id", "count"),
    销售额=("amount", "sum"),
    客单价=("amount", "mean")).round(2)

print(g)
```

分组聚合结果

store	count	sum	mean	max
A店	412	23.20万	563.01	9860
B店	358	20.10万	561.52	8420
C店	388	22.40万	577.38	9980

count 之和 = 1158, 与 amount 非空条数一致

agg() 支持一次求出多个统计量，并可用"名称=(列名, 函数)"自定义结果列名

合并与透视

四种操作方式对比

- 堆叠合并 `concat`: 不关心键, 只按轴拼接, 适合结构相同的多段数据汇总
- 主键合并 `merge / join`: 按 `key` 对齐, 适合把不同字段的表横向关联起来
- 重叠合并 `combine_first`: 两张表信息几乎重复但各有缺口, 互相补齐
- 数据透视 `pivot_table`: 不是"合并表", 而是"重塑表", 把长表转成行列交叉的宽表

选型口诀: 结构相同用 `concat`, 字段互补用 `merge`, 互相缺口用 `combine_first`, 换个视角用 `pivot`

本节小结

合并与透视的四条铁律

- 合并前先对齐：时间范围、时间粒度、时间格式、时区，以及字段命名
- 合并后必查：shape 是否符合预期、是否产生大量 NaN、主键有没有重复
- concat 看 join，merge 看 how —— 这两个参数决定了最终保留哪些行
- 透视表四要素：index 定行、columns 定列、values 定值、aggfunc 定算法

合并的本质是"对齐"，而对齐的前提是"口径一致"

小结

Chapter Summary

1

读取数据

CSV、Excel、数据库三类数据源的读写函数与关键参数

2

数据概览与统计

shape / info / describe / dtypes, 先看清全貌再动手

3

数据清洗

重复值、缺失值、异常值的检测与处理, 以及一致性校验

4

数据合并

堆叠、主键、重叠三种合并方式的选择与实现

5

数据透视表

pivot_table 与 groupby 重塑数据视角, 为可视化做准备

课堂练习

启动Jupyter Notebook，新建一个.ipynb文件，编写代码，调试并运行结果。

基础

读取 data/sales_2026.csv，打印 shape、info() 和前 5 行，并统计每列缺失值个数与占比

基础

把 order_time 列转为 datetime64，再分别按年、月、星期统计订单量

进阶

用箱线图找出 amount 列的异常值，分别用"删除"和"截断"两种方式处理，对比 describe() 结果的差异

进阶

用 pivot_table 生成"门店 × 商品类别"的销售额透视表，并显示合计行列

挑战

把 sales_2026 与商品信息表按 product_id 合并，统计每个类别的客单价并按降序排列